

validation and test sets are constructed with a fixed seed.

We use a GRU layer over the input sequence and take the last-hidden state, in order to get the summary $\mathbf{c}_t$ of the input sequence. The $\mathbf{w}_t$, $\mathbf{l}_t$ are only conditioned on $\mathbf{c}_t$, and the MLP predicting the d_t is conditioned on the latent representations of $\mathbf{w}_t$ and $\mathbf{l}_t$. We use minibatches of size 250 using adam adaptive learning rate algorithm (Kingma and Adam, 2015) using the learning rate of 8×10^{-4} and hidden layers with 1000 units.

We train a model with pointer softmax where we assign pointers for the rarest 60 words and the rest of the words are predicted from the shortlist softmax of size 540. We observe that increasing the inverse temperature of the sigmoid output of d_t to 2, in other words making the decisions of d_t to become sharper, works better, i.e. $d_t = \sigma(2x)$.

At the end of training with pointer softmax we obtain the error rate of 17.4% and by using softmax over all 600 tokens, we obtain the error-rate of 48.2%.

5.2 Summarization

In these series of experiments, we use the annotated Gigaword corpus as described in (Rush et al., 2015). Moreover, we use the scripts that are made available by the authors of (Rush et al., 2015)¹ to preprocess the data, which results to approximately 3.8M training examples. This script generates about 400K validation and an equal number of test examples, but we use a randomly sampled subset of 2000 examples each for validation and testing. We also have made small modifications to the script to extract not only the tokenized words, but also system-generated named-entity tags. We have created two different versions of training data for pointers, which we call *UNK*-pointers data and entity-pointers data respectively.

For the *UNK*-pointers data, we trim the vocabulary of the source and target data in the training set and replace a word by the UNK token whenever a word occurs less than 5 times in either source or target data separately. Then, we create pointers from each UNK token in the target data to the position in the corresponding source document where the same word occurs in the source, as seen in the data before UNK’s were created. It is possible that the source can have an UNK in the matching posi-

tion, but we still created a pointer in this scenario as well. The resulting data has 2.7 pointers per 100 examples in the training set and 9.1 pointers rate in the validation set.

In the entity-pointers data, we exploit the named-entity tags in the annotated corpus and first anonymize the entities by replacing them with an integer-id that always starts from 1 for each document and increments from left to right. Entities that occur more than once in a single document share the same id. We create the anonymization at token-level, so as to allow partial entity matches between the source and target for multi-token entities. Next, we create a pointer from the target to source on similar lines as before, but only for exact matches of the anonymized entities. The resulting data has 161 pointers per 100 examples in the training set and 139 pointers per 100 examples in the validation set.

If there are multiple matches in the source, either in the UNK-pointers data or the entity-pointers data, we resolve the conflict in favor of the first occurrence of the matching word in the source document. In the UNK data, we model the UNK tokens on the source side using a single placeholder embedding that is shared across all documents, and in the entity-pointers data, we model each entity-id in the source by a distinct placeholder, each of which is shared across all documents.

In all our experiments, we use a bidirectional GRU-RNN (Chung et al., 2014) for the encoder and a uni-directional RNN for the decoder. To speed-up training, we use the large-vocabulary trick (Jean et al., 2014) where we limit the vocabulary of the softmax layer of the decoder to 2000 words dynamically chosen from the words in the source documents of each batch and the most common words in the target vocabulary. In both experiments, we fix the embedding size to 100 and the hidden state dimension to 200. We use pre-trained word2vec vectors trained on the same corpus to initialize the embeddings, but we finetune them by backpropagating through the pre-trained embeddings during training. Our vocabulary sizes are fixed to 125K for source and 75K for target for both experiments.

We use the reference data for pointers for the model only at the training time. During the test time, the switch makes a decision at every timestep on which softmax layer to use.

¹<https://github.com/facebook/NAMAS>